



Relaciones entre Clases en Java

Chuletario · Unidad 8 · DAM

Asociación

Agregación

Composición

Herencia

Relación	Tipo	Ejemplo
• Asociación	Débil	Empleado–Empresa
• Agregación	Media	Coche–Motor
• Composición	Fuerte	Persona–Corazón
• Herencia	Muy fuerte	Perro–Animal



Asociación

"Se conocen" · HAS-REFERENCE · Relación débil

Dos clases se relacionan pero son **independientes**.
Ninguna crea a la otra. → "trabaja con", "usa"

1A1 EJEMPLO BÁSICO

```
class Empresa {
    private String nombre;
    public Empresa(String nombre) {
        this.nombre = nombre;
    }
    public String getNombre() { return nombre; }
}

class Empleado {
    private String nombre;
    private Empresa empresa; // ← asociación
    public Empleado(String nombre, Empresa empresa) {
        this.nombre = nombre;
        this.empresa = empresa;
    }
    public void mostrarInfo() {
        System.out.println(nombre + " trabaja en "
            + empresa.getNombre());
    }
}
```

1A N UNO A MUCHOS · ARRAYLIST

```
class Empresa {
    private List<Empleado> empleados = new ArrayList<>();
    public void agregarEmpleado(Empleado e) {
        empleados.add(e);
    }
}
```

- ✓ Ambas clases existen de forma **independiente**
- ✓ El objeto se **pasa como parámetro**
- ✓ No hay dependencia de ciclo de vida



Agregación

"HAS-A débil" · Contiene pero no crea · Parte vive sola

Una clase **contiene** a otra, pero ambas pueden existir por separado.
→ "tiene un" (independiente)

EJEMPLO COCHE TIENE MOTOR

```
class Motor {
    private int potencia;
    public Motor(int potencia) {
        this.potencia = potencia;
    }
    public int getPotencia() { return potencia; }
}

class Coche {
    private String marca;
    private Motor motor; // ← agregación
    // Motor se crea FUERA y se pasa
    public Coche(String marca, Motor motor) {
        this.marca = marca;
        this.motor = motor;
    }
    public void mostrarInfo() {
        System.out.println("Coche: " + marca
            + " con " + motor.getPotencia() + " CV");
    }
}

// En Main:
Motor m = new Motor(150); // Motor independiente
Coche c = new Coche("Toyota", m); // se le pasa
```

- ✓ El objeto **se crea fuera** y se inyecta
- ✓ La parte **puede reutilizarse** en otras clases
- ✓ Destruir el todo **no destruye** la parte

 Composición

"HAS-A fuerte" · La parte NO existe sin el todo

Una clase **crea** internamente a la otra.
La parte **no puede existir** sin el todo. → relación fuerte

EJEMPLO

PERSONA TIENE CORAZÓN

```
class Persona {
    private String nombre;

    // Clase interna privada → solo existe aquí
    private class Corazon {
        public void latir() {
            System.out.println("El corazón late");
        }
    }

    private Corazon corazon;

    public Persona(String nombre) {
        this.nombre = nombre;
        this.corazon = new Corazon(); // ← creado aquí
    }

    public void vivir() {
        System.out.println(nombre + " está vivo");
        corazon.latir();
    }
}
```

AGREGACIÓN VS COMPOSICIÓN

 **Agregación**

Objeto se *pasa* por parámetro
Parte puede existir sola
Relación *débil*

 **Composición**

Objeto se *crea dentro*
Parte no existe sola
Relación *fuerte*

- ✓ La parte se **crea dentro** del constructor
- ✓ Se destruye cuando se destruye el todo
- ✓ Suele implementarse como **clase interna**

 Herencia

"ES UN" · extends · Polimorfismo

Una clase **es un tipo de** otra. Hereda atributos y métodos.
→ extends

```
class Animal {
    protected String nombre;
    public Animal(String n) { this.nombre = n; }
    public void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {
    public Perro(String n) { super(n); }
    @Override
    public void hacerSonido() {
        System.out.println("Guau");
    }
}

// Polimorfismo:
Animal a = new Perro("Firulais");
a.hacerSonido(); // → "Guau" (Perro override)
```

- ✓ Usa `super()` para llamar al constructor padre
- ✓ **Polimorfismo** : tipo padre apunta a hijo
- ✓ Hereda todo lo `public` y `protected`

 @Override

Sobrescribir método de la superclase

Mismo nombre, mismos parámetros, mismo tipo de retorno.
La subclase **reemplaza** la implementación del padre.

```
public abstract class Factura {
    private double importe;
    public double getImporte() { return importe; }
    public abstract double calcularTotal();
}

class FacturaConIva extends Factura {
    @Override
    public double calcularTotal() {
        return getImporte() * 1.21; // +21% IVA
    }
}

class FacturaSinIva extends Factura {
    @Override
    public double calcularTotal() {
        return getImporte(); // sin IVA
    }
}
```

 ¿Cuándo usar cada una?

Guía de decisión rápida

 Usa **ASOCIACIÓN** cuando...

Las clases solo se *relacionan* o se *conocen*
→ Cliente – Pedido | Profesor – Asignatura

 Usa **AGREGACIÓN** cuando...

Una clase *tiene* otra, pero pueden *vivir separadas*
→ Coche – Motor | Equipo – Jugador

 Usa **COMPOSICIÓN** cuando...

La parte *no tiene sentido* sin el todo
→ Casa – Habitaciones | Pedido – Líneas

 Usa **HERENCIA** cuando...

Hay relación *"ES UN"* real y clara
→ Perro – Animal | Coche – Vehículo

TABLA RESUMEN

Relación	Palabra clave	Vida independiente	Fuerza
● Asociación	"usa / trabaja con"	✓ Sí	Débil
● Agregación	"tiene un"	✓ Sí	Media
● Composición	"es parte de"	X No	Fuerte
● Herencia	"es un"	X No	Muy fuerte

REGLA RÁPIDA

¿Solo se conocen? → Asociación
¿Una tiene a la otra (creada fuera)? → Agregación
¿Una crea a la otra dentro? → Composición
¿Una es la otra? → Herencia